GrafanaCON '26

Getting started with OpenTelemetry instrumentation: A practical playbook

Mario Macías
Principal Software Engineer

Liudmila Molkova
Staff Developer Advocate

GrafanaCON '26

We're here to help!

Liudmila

Mario

Bejal

Marko

Nikola

Yoshi

Agenda

Section	Time
Introduction to OpenTelemetry	20 min
Collecting infra metrics	30 min
Collecting baseline metrics with OBI	30 min
Instrumenting applications	40 min
Customizing instrumentation	20 min
Processing telemetry	20 min
[Stretch] Live demo: adding manual instrumentation	10 min
Q&A and Wrap up	10 min

https://tinyurl.com/gcon2026-otel

Introduction to OpenTelemetry

What is OpenTelemetry?

Open-source observability framework

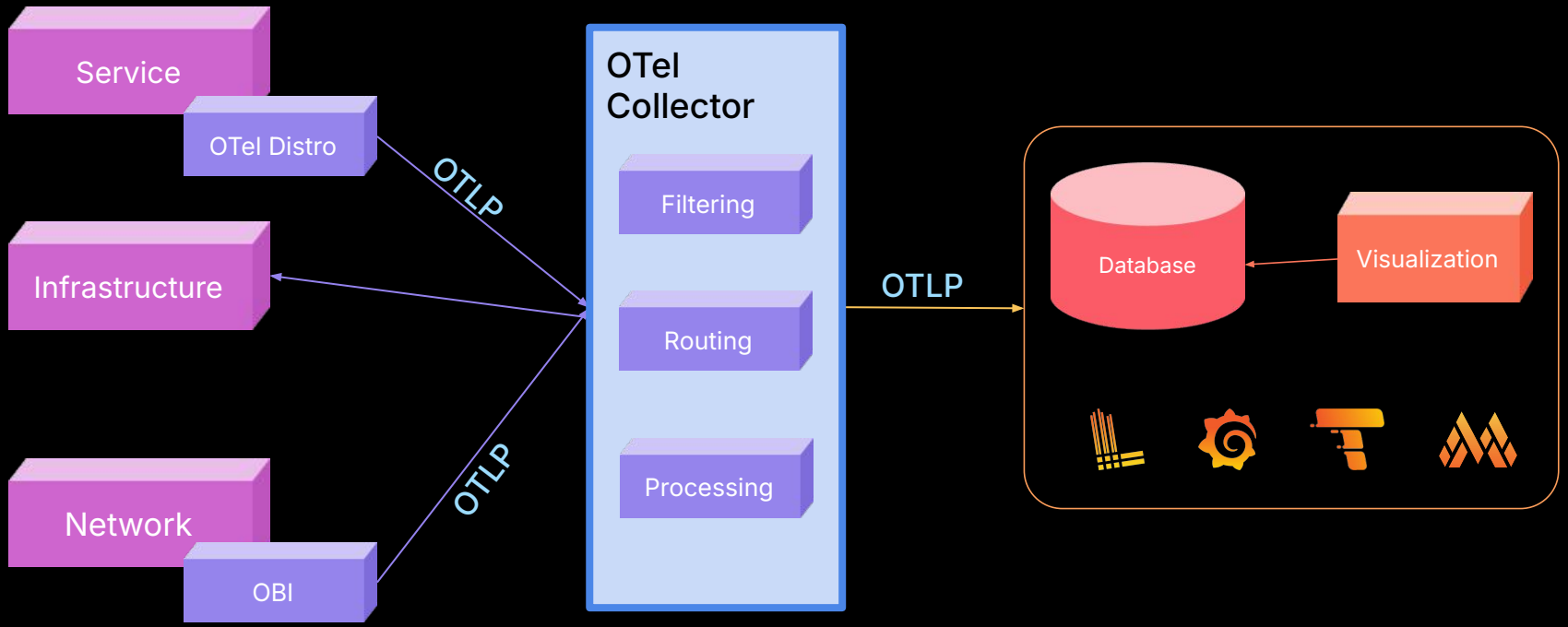
- Generate and export telemetry with OTel SDK
- OpenTelemetry Collector for telemetry ingestion, processing, and routing
- Unified telemetry following Semantic Conventions

Industry standard

- Supported by all major observability vendors
- Vendor-agnostic: instrument once, use with any vendor
- CNCF-governed, second most active project after Kubernetes

https://opentelemetry.io/

Core components



OTel Telemetry Signals

Spans & Traces

Span represents a **single operation** with **duration**, attributes, and **status**.

Metrics

Aggregated measurements that track system behavior **over time** .

Logs

Timestamped records of events that occurred in a system.

Profiles

Alpha stage

Show where an application **spends time or uses resources** (e.g., CPU).

Semantic Conventions

- **Standardize** attribute and signal **names** for technologies like HTTP, databases, GenAI, messaging, cloud resources and more
- Help ensure telemetry is **consistent** across services, languages, and instrumentations
- Make data **easier to understand** and **query** across observability tools

https://opentelemetry.io/docs/specs/semconv/

OTLP – OpenTeLemetry Protocol

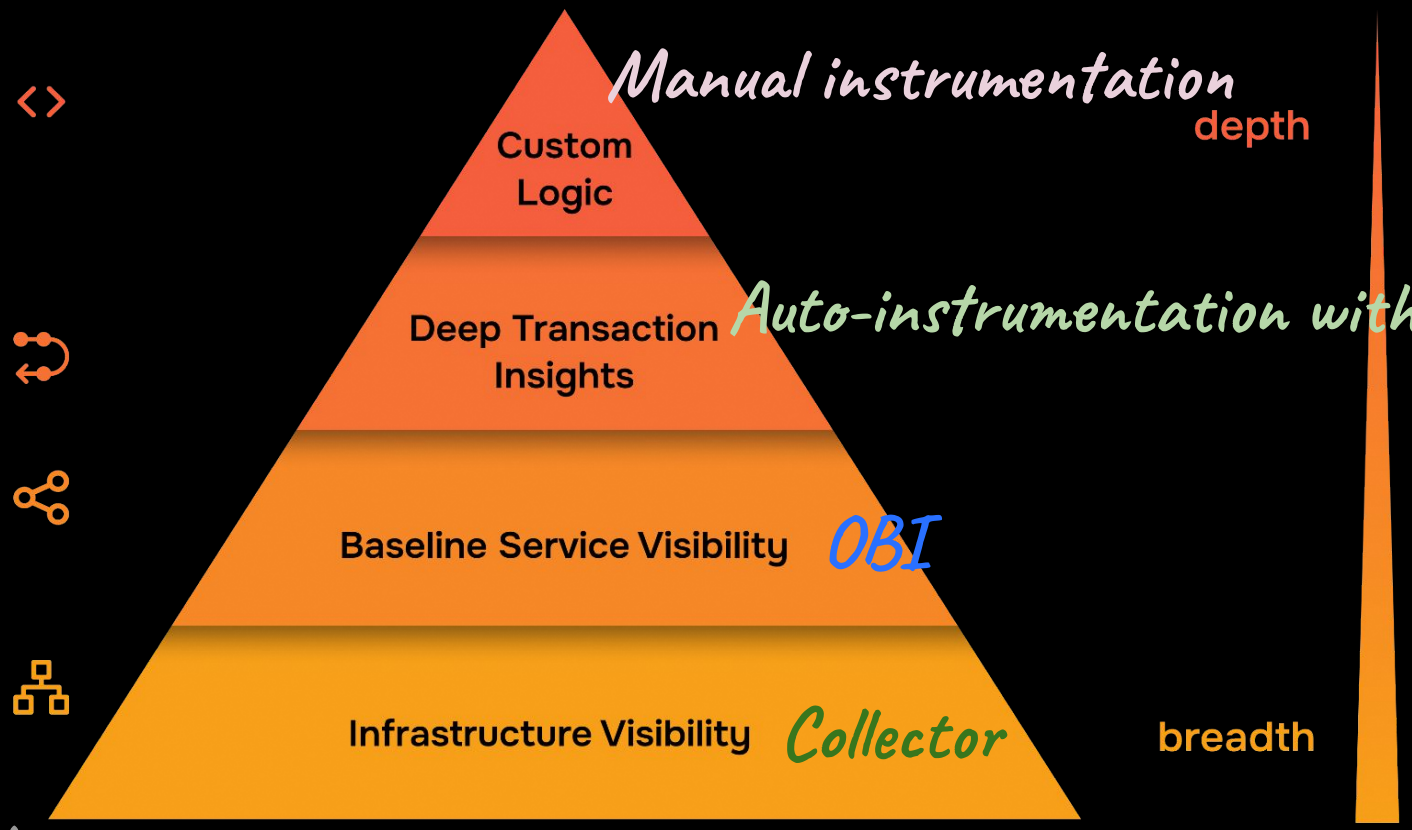
- **Over the wire format** supported by most vendors
- **Formal representation** of telemetry signals
- **gRPC or HTTP** under the hood

https://github.com/open-telemetry/opentelemetry-proto


```
{ "resourceSpans": [ {
    "resource": {
      "attributes": [
        { "key": "service.name", "value": { "stringValue": "frontend"
    ]},
    "scopeSpans": [ {
        "scope": {"name": "@opentelemetry/instrumentation-http" },
        "spans": [ {
            "traceId": "5b8efff798038103d269b633813fc60c",
            "spanId": "eee19b7ec3c1b174",
            "parentSpanId": "eee19b7ec3c1b173",
            "name": "GET /restaurants/{id}",
            "startTimeUnixNano": "1544712660000000000",
            "endTimeUnixNano": "1544712661000000000",
            "kind": 2,
            "attributes": [
              {
                "key": "http.request.method",
                "value": { "stringValue": "GET" }
              },
              {
                "key": "url.full",
                "value": { "stringValue": "http://localhost:8080/rest
              }
            ]},
```

The instrumentation pyramid of needs

The instrumentation pyramid of needs



Observability as a feature of your services

Observability as an infrastructure feature

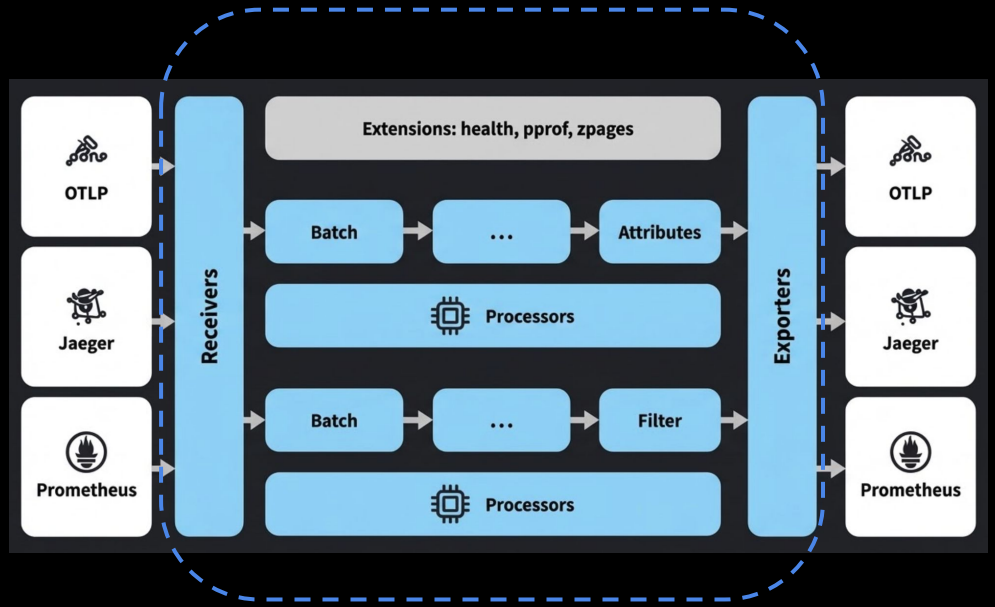
Collecting infra metrics

Common Infrastructure Signals

- **Host / Node:** CPU, GPU, memory, network, disc, etc
- **K8s: Pod / container** resource usage, restarts, scheduling, scaling
- **Databases**: read/write throughput, latency, replication, cache hit ratio, execution plans
- **Load balancers and ingress**: HTTP logs
- **Cloud**: auto-scaling events, rate limits, access and activity logs

and many more ...

OpenTelemetry Collector



- **Receive** and **export** telemetry in OTLP and other formats
- Collect **infra metrics**
- Enrich telemetry with environment details
- **Sample**, **drop**, **sanitize**, and **transform** telemetry

https://github.com/open-telemetry/opentelemetry-collector-contrib

Example of collector-config.yaml

```yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

  docker_stats:
    endpoint: unix:///var/run/docker.sock

  hostmetrics:
    root_path: /hostfs
    scrapers:
      cpu: {}

processors:
  resourcedetection:
    detectors: [env, system, docker]
  batch:
```

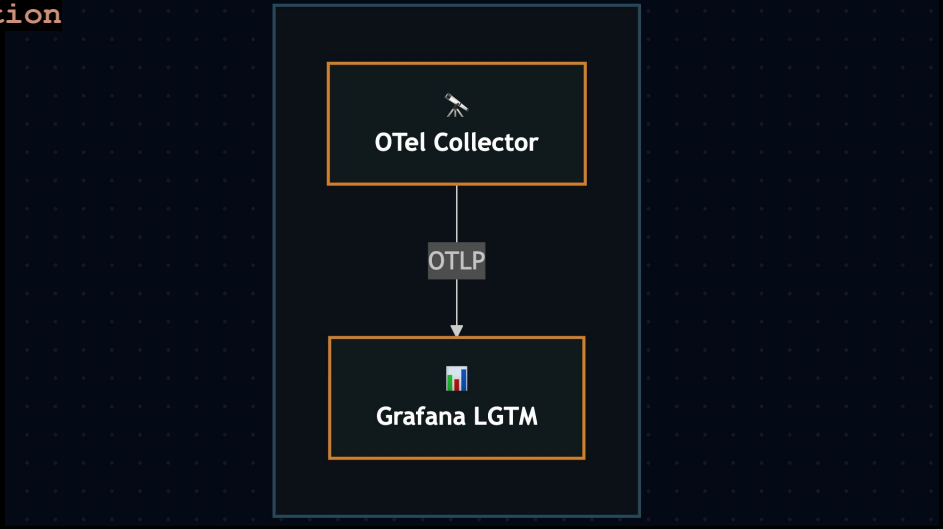
```yaml
exporters:
  otlp_http:
    endpoint: http://lgtm:4318

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [resourcedetection, batch]
      exporters: [otlp_http]
    metrics:
      receivers: [otlp, docker_stats, hostmetrics]
      processors: [resourcedetection, batch]
      exporters: [otlp_http]
    logs:
      receivers: [otlp]
      processors: [resourcedetection, batch]
      exporters: [otlp_http]
```

Exercise: enable basic infra metrics

Collecting infra metrics

Setup

```
$ git clone https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation
$ cd grafanacon2026-opentelemetry-instrumentation
$ docker compose up --build
```



Check:

- App at http://localhost:8080
- Grafana at http://localhost:3000

https://tinyurl.com/gcon2026-otel

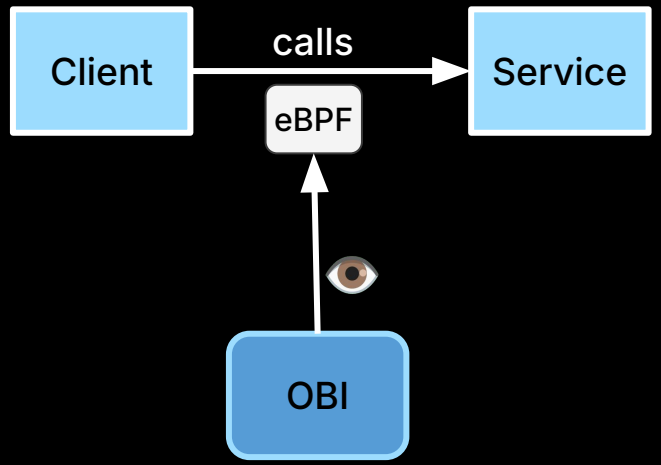
Goal: Collect infra metrics via OTel Collector

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation/blob/main/exercises/01-setup-infra-metrics.md

https://tinyurl.com/gcon2026-otel

Collecting baseline service metrics with OBI

OBI – OpenTelemetry eBPF Instrumentation



- Intercepts calls to **Linux Kernel**
- Understands common network protocols
- Can instrument **any application** on modern Linux, **without** code changes or even **restart**
- Provides **baseline observability** - request and error rates, duration between endpoints (**RED metrics**)

https://github.com/open-telemetry/opentelemetry-ebpf-instrumentation

Exercise: setup OBI

Collecting baseline service metrics with OBI

Create obi/obi-config.yaml

```yaml
otel_metrics_export:
  protocol: http/protobuf
  endpoint: http://otel-collector:4318
  interval: 5s
discovery:
 instrument:
  - container_name: backend
  - container_name: frontend
```

Add to docker-compose.yaml

```yaml
obi:
  container_name: obi
  image: otel/ebpf-instrument
  privileged: true
  pid: host
  volumes:
    - ./obi/obi-config.yaml:/etc/obi/config.yaml:ro
  environment:
    OTEL_EBPF_CONFIG_PATH:
/etc/obi/config.yaml
```

Goal: Collect RED metrics with OBI

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation/blob/main/exercises/02-setup-obi.md

https://tinyurl.com/gcon2026-otel

Coll

G CS

Dashboards › RED Metrics

Search... ⌘+k

Sig

Edit | Export | Share

Data source: Prometheus | Service: All | Job: All | Instance: All

Last 30 minutes | Refresh 30s

Slowest HTTP routes (P95)

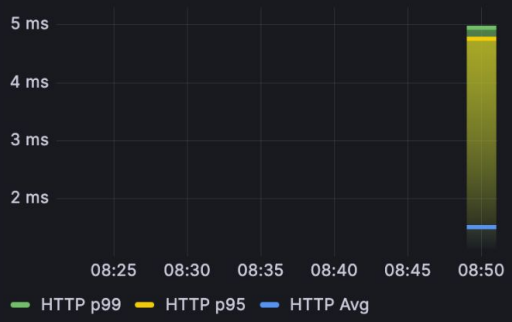
service_name	http_route	Duration (ms)
frontend	/search	10 s s
frontend	/*	5 s s
backend	/api/restaurants	5 s s

Slowest RPC methods (P95)

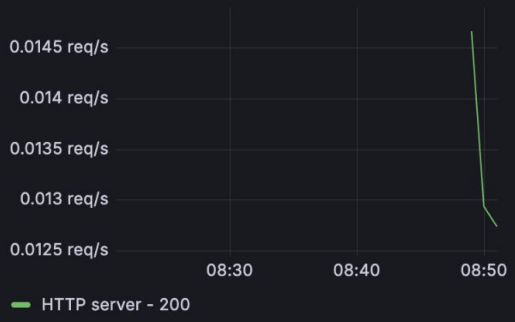
No data

Inbound: backend

Duration



Request rate



Error rate

No data

Inbound: db

Duration | Request rate | Error rate

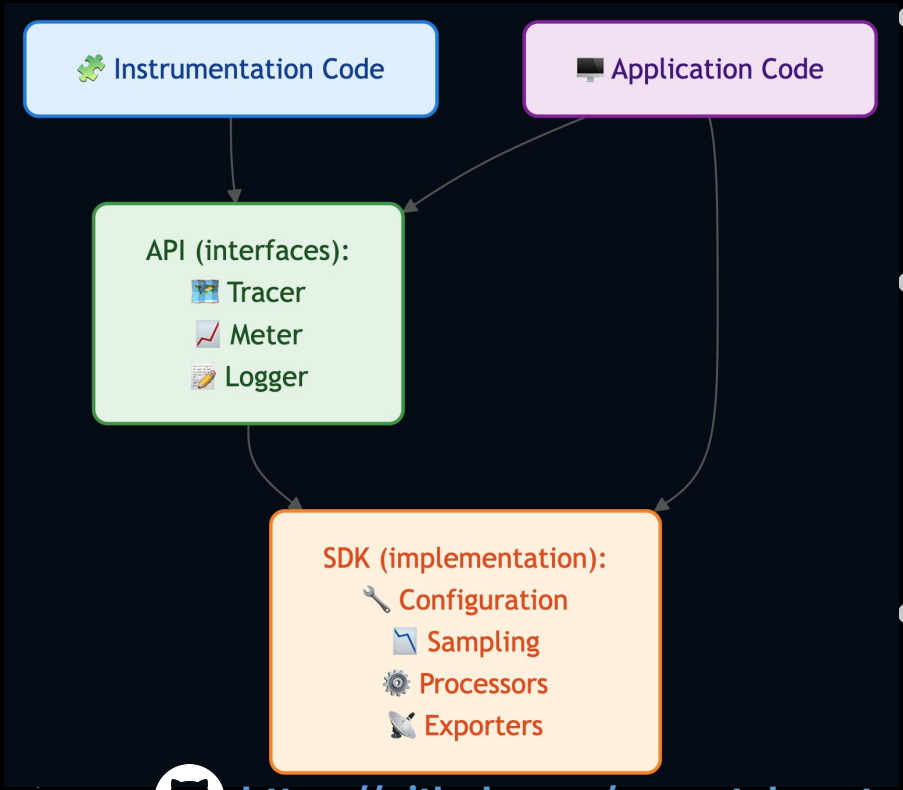
Instrumenting applications

OTel Distributions

```
OTEL_EXPORTER_OTLP_ENDPOINT="otel-collector:4318" \
OTEL_SERVICE_NAME="frontend" \
NODE_OPTIONS="--require @opentelemetry/auto-instrumentations-node/register" \
node frontend.js
```

- **All you need** to instrument a typical app in a given language
- **Zero or minimal code changes** to onboard
- Includes OTel API, SDK, and **common instrumentations**
- **Configurable** and **customizable**

OTel API and SDK



- **Instrumentations** - Produce telemetry using the API. The API and SDK themselves do not collect telemetry
- **OTel API** - defines instrumentation interfaces for libraries and applications. Lightweight and stable. Does nothing without an SDK.
- **OTel SDK** – implements behavior, configuration, and telemetry export.

https://github.com/open-telemetry/opentelemetry-specification

Zero-code onboarding

The OTel SDK and instrumentations are **injected at startup** via language-specific mechanisms. This is available for most languages.

```
$ npm install @opentelemetry/auto-instrumentations-node
$ OTEL_EXPORTER_OTLP_ENDPOINT="otel-collector:4318"                          \
  OTEL_SERVICE_NAME="frontend"                                               \
  NODE_OPTIONS="--require opentelemetry/auto-instrumentations-node/register" \
  node frontend.js
```

Onboarding with declarative config

Works with manual or zero-code onboarding.
Reduce boilerplate with language-agnostic config.
Supported in some languages, more to come

```java
package otel;

import io.opentelemetry.sdk.OpenTelemetrySdk;
import io.opentelemetry.sdk.autoconfigure.AutoConfiguredOpenTelemetrySdk;

public class AutoConfiguredSdk {
  public static OpenTelemetrySdk autoconfiguredSdk() {
    return AutoConfiguredOpenTelemetrySdk.initialize().getOpenTelemetrySdk();
  }
}
```

```yaml
# OTEL_CONFIG_FILE=/path/to/otel-sdk-config.yaml
# otel-sdk-config.yaml

file_format: "1.0"

resource:
  detection/development:
    detectors:
      - service:

propagator:
  composite:
    - tracecontext:
    - baggage:

tracer_provider:
  sampler:
    parent_based:
      root:
        always_on:
  processors:
    - batch:
        exporter:
          otlp_http:
            endpoint: ${OTEL_EXPORTER_OTLP_ENDPOINT}

meter_provider:
  readers:
    - periodic:
        exporter:
          otlp_http:
            endpoint: ${OTEL_EXPORTER_OTLP_ENDPOINT}

logger_provider:
  processors:
    - batch:
        exporter:
          otlp_http:
            endpoint: ${OTEL_EXPORTER_OTLP_ENDPOINT}
```

Configuration

- **Environment variables:** service name, resource attributes, export endpoints, samplers, enable/disable instrumentations, language-specific options
- **Declarative config** (recommended), everything above plus **pipeline configuration**
- **Programmatic config**: full control over configuration and extensibility points

Instrumentations

- **Auto-instrumentations:** don’t require app to change code.
- **Hybrid:** enabled with a few lines of code at app startup
- **Manual:** written by application developers for their own needs

Instrumentations - where to find them

- `opentelemetry-{language}-contrib` or `opentelemetry-{language}-instrumentation` repositories
- Ecosystem Explorer - https://explorer.opentelemetry.io/
- OpenTelemetry Registry - https://opentelemetry.io/ecosystem/registry/
- Library documentation for native and first-party instrumentations

Exercise: Enable application instrumentation

Goal: Collect application-level telemetry

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation/blob/main/exercises/03-instrumenting-applications.md

https://tinyurl.com/gcon2026-otel

Customizing app instrumentation

Typical customization needs

Cost control

Suppress or sample spans and logs

Drop metric attributes

Enrichment

Add application-specific context

Privacy and security

Anonymize user info

Sanitize sensitive information

Testing instrumentation

In-memory exporters

Customization points

SDK Configuration (env vars, yaml)

Enable / disable / customize auto-instrumentations

Select head-based sampler and sampling probability

Set log verbosity level

Configure metric aggregation

Application code

Custom samplers / processors

Enrichment

Custom instrumentation

Collector

Drop metrics / logs / spans and their attributes

Tail sampling

Spans / logs → metrics

Normalization, sanitization

Routing

Example: Cost control with sampling

Head-based sampling in the SDK:

- Controls **% of traces** collected
- Usually **probabilistic** (configured via env vars or config)
- Custom samplers / filters to **drop noisy spans** (e.g., static files, health checks)

Collector can apply **tail-based sampling** to keep only interesting traces (errors, slow requests)
https://opentelemetry.io/docs/concepts/sampling/#tail-sampling

Example: enrichment with app-specific context

Auto-instrumentation captures **generic details**, not application context. To add unique context like user / session and other app-specific details

- Add them **to active span** in middleware or incoming request handler
- Use **span and log processors** to inject properties **from active context**. This applies to all spans and logs

Where to apply customizations

SDK configuration / code

App context: capture business logic and app-specific details

Low flexibility: requires app deployment / restart

Controlled overhead: better not to collect data than to drop it later

OTel Collector

Environment context: capture infra signals and details

High flexibility: can change dynamically, no app restart

Universal pipeline: receive anything, filter/transform as needed

Centralized control: verbosity, security, compliance controlled by platform team

Exercise: Customize telemetry collection

Goal: Drop noise and enrich telemetry

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation/blob/main/exercises/04-customizing-instrumentations.md

https://tinyurl.com/gcon2026-otel

Processing telemetry

Processing telemetry in Collector with OTTL

- OTTL (**OpenTelemetry Transformation Language**) enables **transformations** for traces, metrics, logs, and profiles
- Apply transformations **in-flight** within the telemetry pipeline
- Common uses: modifying attributes, **normalizing** telemetry, and **redacting** sensitive fields
- Implemented through **processors** such as transform and filter

https://opentelemetry.io/docs/collector/transforming-telemetry/

https://ottl.run/

Exercise: processing telemetry

Goal: Sanitize telemetry using OTTL in the Collector

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation/blob/main/exercises/05-processing.md

https://tinyurl.com/gcon2026-otel

[Stretch] Live demo: adding manual instrumentation

Goal: Learn how to write high-quality instrumentation

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation/blob/main/exercises/06-manual-instrumentation.md

https://tinyurl.com/gcon2026-otel

Wrap up

Wrap up

Summary

- OpenTelemetry is a **vendor-neutral** framework for collecting **metrics, logs, traces, and profiles**.
- Follow the Instrumentation Pyramid: start with **infrastructure**, add baseline **network** visibility with OBI, then **deep insights** via OTel SDK and instrumentations.
- **Customize collection**: Use SDK or Collector sampling/enrichment to **control costs** and maximize observability.
- Grafana ❤️ OTel: The **LGTM stack** ingests OTLP data to show all telemetry in one place.

Q&A

Take home materials & what to check out at GrafanaCON

https://github.com/grafana/grafanacon2026-opentelemetry-instrumentation

Check out:

- **Tuesday, 14:00: Alloy's OpenTelemetry engine: What's new and what's next**
- **Wednesday, 8:00: OpenTelemetry and Alloy group discussion**

Take our survey!

Help us improve! Show us your completion screen and get a pin!